

1. CONOCIMIENTOS SQL

1.1) Describe el funcionamiento general de la sentencia JOIN.

Sirve para juntar dos o más tablas que estén relacionadas entre una llave primaria y una llave foránea, y con esto consultar los datos en conjunto.

1.2) ¿Cuáles son los tipos de JOIN y cuál es el funcionamiento de los mismos?

INNER JOIN: Es el join principal, que devuelve los datos donde existe una coincidencia en la llave primaria y la foránea; los demás datos no se muestran.

LEFT JOIN: Devuelve todos los datos de la primera tabla mencionada en la consulta sin importar que coincida con la segunda tabla. De esta segunda solo se muestran los datos que coinciden con la primera, y en los datos donde no tiene coincidencia se muestra el valor NULL.

RIGHT JOIN: Tiene la misma función que el Left pero invertido, ya que en esta muestra de la primera tabla mencionada solo los datos que coinciden con la segunda, y de esta segunda muestra todos sus datos sin importar que tenga coincidencia o no. Al igual que en el Left, en los datos que no coincide se muestra con valor NULL.

FULL JOIN: Devuelve todos los datos de ambas tablas, tengan o no coincidencia. Para usarlo en MySQL se utiliza la sentencia UNION entre el LEFT JOIN y el RIGHT JOIN.

1.3) ¿Cuál el funcionamiento general de los TRIGGER y qué propósito tienen?

Su función general es establecer reglas necesarias desde la base de datos para no cargarle ese peso al Backend y automatizar la ejecución de esas reglas, ya que se ejecutan automáticamente en dos momentos que son BEFORE y AFTER en los eventos INSERT, UPDATE y DELETE.

Estos sirven para automatizar los procesos internos y para asegurar la integridad de la información para no depender de que el Backend ejecute múltiples consultas.

1.4) ¿Qué es y para qué sirve un STORED PROCEDURE?

Es un conjunto de instrucciones o consultas guardadas en la base de datos que son llamadas con el comando CALL

Estos sirven para encapsular múltiples operaciones en un solo procedimiento para aligerar el tráfico de las aplicaciones que necesitan realizar varias peticiones a la base de datos y estas ayudan a prevenir los ataques de inyección SQL.

1.5) Traer todos los productos que tengan una venta.

```
41  -- 1.5) Traer todos los productos que tengan una venta.
42  • SELECT DISTINCT p.idProducto, p.nombre
43  FROM productos p
44  INNER JOIN ventas v ON p.idProducto = v.idProducto;
45
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	idProducto	nombre
▶	1	LAPTOP
	2	PC
	4	TECLADO
	5	MONITOR
	6	MICROFONO

1.6) Traer todos los productos que tengan ventas y la cantidad total de productos vendidos.

```
47  -- 1.6) Traer todos los productos que tengan ventas y la cantidad total de productos vendidos.
48  • SELECT p.idProducto, p.nombre, SUM(v.cantidad) AS total_vendido
49  FROM productos p
50  INNER JOIN ventas v ON p.idProducto = v.idProducto
51  GROUP BY p.idProducto, p.nombre;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	idProducto	nombre	total_vendido
▶	5	MONITOR	9
	1	LAPTOP	23
	6	MICROFONO	19
	2	PC	8
	4	TECLADO	5

1.7) Traer todos los productos (independientemente de si tienen ventas o no) y la suma total (\$) vendida por producto.

```

54  -- 1.7) Traer todos los productos (independientemente de si tienen ventas o no)
55  --      y la suma total ($) vendida por producto.
56  • SELECT p.idProducto, p.nombre, IFNULL(SUM(p.precio * v.cantidad), 0) AS total_dinero
57  FROM productos p
58  LEFT JOIN ventas v ON p.idProducto = v.idProducto
59  GROUP BY p.idProducto, p.nombre;

```

Result Grid

	idProducto	nombre	total_dinero
▶	1	LAPTOP	69000.00
	2	PC	32000.00
	3	MOUSE	0.00
	4	TECLADO	750.00
	5	MONITOR	18000.00
	6	MICROFONO	6650.00
	7	AUDIFONOS	0.00

2.1) Crea un diagrama relacional de BD para el escenario descrito anteriormente.

